```
      marketId: String,
      odds: Double,
      stake: Int,
      result: Int,
      session: ScalikeJdbcSession): Unit = {
    session.db.withinTx { implicit dbSession =>
      sql"""
           INSERT INTO                                                    ❶
              bet_wallet_market (betId, walletId, marketId, odds, stake, result)
              VALUES ($betId, $walletId, $marketId, $odds, $stake, $result)
            ON CONFLICT (betId) DO NOTHING""".executeUpdate().apply()
    }

  }

  override def getBetPerMarketTotalStake(
      marketId: String,
      session: ScalikeJdbcSession): List[StakePerResult] = {
    session.db.readOnly { implicit dbSession =>
      sql"""SELECT sum(stake * odds), result FROM bet_wallet_market      ❷
                  WHERE marketId = $marketId GROUP BY marketId, result"""
        .map(rs => StakePerResult(rs.double("sum"), rs.int("result")))
        .list
        .apply()
    }
  }
}
```

❶ Creates entries on bet_wallet_market

❷ Queries the total per market per result

This repository implementation is then used by the handler of the projection, which links the events from the journal to the upsert.

Listing 15.2 `BetProjectionHandler`

```
class BetProjectionHandler(repository: BetRepository)
    extends JdbcHandler[EventEnvelope[Bet.Event], ScalikeJdbcSession] {    ❶

  val logger = LoggerFactory.getLogger(classOf[BetProjectionHandler])

  override def process(
      session: ScalikeJdbcSession,                                         ❷
      envelope: EventEnvelope[Bet.Event]): Unit = {                        ❸
    envelope.event match {
      case Bet.Opened(betId,walletId,marketId,odds,stake,result) =>        ❹
```